

Language Detection Package Design



I want to create a java package for the purpose of language detection. For that, I want an initial structure of this project (including sub-packages, files and classes, and their hierarchical location in the project folder). Assuming that this package does not need to implement AI (i.e. no Deep Learning concepts applied), thus, I wanted a naïve rules, function and methods for language detection. The concept is the following:

In the overall view, I want a global mechanism (e.g. function, class, or method, ...etc.), that takes Arraylike of tokens (i.e. a list of text), and returns a classification of the three languages; whether English, French, or Arab (only restricted to those three classes). But, as obvious (not just as simple function as it looks like), you'll need extra-libraries, or tools that helps this mechanism, for achieving this purpose.

Now, what is required for you to do now, is only reading, understanding, analyzing, and identifying relevant points of this statement.



Got it! Here's a summary of your requirements and relevant points for developing the project:

1. Language Detection Goal:

- Classify tokens (text samples) as English, French, or Arabic.
- Use simple, rule-based methods for classification rather than AI or deep learning.

2. Structure Requirements:

- A project structure to support modularity, clarity, and potential scalability.
- A global mechanism (like a central method or function) that accepts an array-like list of text tokens and returns a language classification.

3. Implementation Details:

- This mechanism may leverage libraries or external tools for handling language-specific nuances (such as regex or specific libraries for detecting alphabets or common words).
- The approach will rely on defining a set of rules or patterns characteristic of English, French, and Arabic. For example, regex patterns to detect common language-specific characters or tokens.

4. Scope:

- The solution should be focused on these three languages only.
- The design should be extensible for future language additions if needed but optimized for the three current languages.

5. Non-AI Approach:

- All methods and functions will use rules, heuristics, or other deterministic approaches for classification, avoiding ML models.

If you'd like, I can propose a package structure with specific classes, methods, and suggested dependencies for you to review.



If you'd like, I can propose a package structure with specific classes, methods, and suggested dependencies for you to review.

Yes! I'd like to get an initial packaging structure of this project, including files, their corresponding hierarchical location, and sub-packages if needed.



Here's an initial packaging structure for your Java language detection project, focusing on clarity and modularity. This structure organizes the logic by separating core detection mechanisms, utilities, and rule definitions.

Project Structure

CSS

```
src/
├── com/
│   └── atx/
│       └── langdetect/
│           ├── detector/
│           │   ├── LanguageDetector.java
│           │   └── Language.java
│           ├── rules/
│           │   ├── EnglishRule.java
│           │   ├── FrenchRule.java
│           │   └── ArabicRule.java
│           └── util/
│               ├── TextUtils.java
│               ├── LanguageDetectionException.java
│               └── LangDetectMain.java
```

Package and Class Descriptions

1. Main Package (`com.atx.langdetect`):

- **Purpose:** Holds the main classes and provides an entry point for testing or initializing the language detection.
- **Class:** `LangDetectMain`
 - This is the entry point to the project, which sets up an example or test case for the language detection system. It may call the `LanguageDetector` class to classify language based on sample input.

2. `detector` Package (`com.atx.langdetect.detector`):

- **Purpose:** Core package for detection logic, where main classes that handle language detection logic reside.
- **Class:** `LanguageDetector`
 - Contains the main detection logic, which accepts a list or array of tokens. It iterates over the tokens, applies rules, and returns the detected language based on the matches

found.

- Uses instances of the rule classes (``EnglishRule``, ``FrenchRule``, ``ArabicRule``) to evaluate each token and identify the language.

- **Enum:** ``Language``

- Defines the supported languages: ``ENGLISH``, ``FRENCH``, and ``ARABIC``. This enum serves as a classification result type for the detector.

3. ``rules`` Package (``com.atx.langdetect.rules``):

- **Purpose:** Holds rule classes specific to each language, making it easier to add new languages by creating new rule classes in the future.
- **Class:** ``EnglishRule``
 - Contains specific rules, patterns, or heuristics for detecting English text, such as checking for common English words or characters.
- **Class:** ``FrenchRule``
 - Implements rules and patterns for detecting French, such as special characters (é, è, ç) or common French words (e.g., "le", "de").
- **Class:** ``ArabicRule``
 - Contains logic for Arabic language detection, which might involve detecting Arabic script Unicode or other unique Arabic language patterns.

4. ``util`` Package (``com.atx.langdetect.util``):

- **Purpose:** Houses utility and helper classes for common functions and error handling.
- **Class:** ``TextUtils``
 - Provides utility methods for processing text, such as removing punctuation, converting text to lowercase, or splitting text into tokens if needed.
- **Class:** ``LanguageDetectionException``
 - A custom exception class to handle errors specific to language detection, like unsupported language tokens or invalid input data.

Description of Core Workflow

1. ``LangDetectMain`` initializes the ``LanguageDetector`` and passes an array of text tokens to it.
2. ``LanguageDetector`` iterates over each token and applies the relevant rule classes (``EnglishRule``, ``FrenchRule``, ``ArabicRule``).
3. Based on matches and thresholds defined in the rule classes, ``LanguageDetector`` determines the most probable language.
4. The result, in the form of a ``Language`` enum, is returned to the main class, which prints or processes it as needed.

This structure allows easy expansion by simply adding new classes in the ``rules`` package for other languages and adjusting the main detection logic accordingly. Let me know if you'd like details on specific methods within these classes or any adjustments to the structure.



****Class: `LanguageDetector`**** * Contains the main detection logic, which accepts a list or array of tokens. It iterates over the tokens, applies rules, and returns the detected language based on the matches found. * Uses instances of the rule classes (`EnglishRule`, `FrenchRule`, `ArabicRule`) to evaluate each token and identify the language.

Give me the "LanguageDetector.java" script, implementing this concepts and logic. Assuming the existence of EnglishRule, FrenchRule, and ArabicRule, this class should use instances of the rule classes.



Here's the initial implementation of the `LanguageDetector.java` class based on the provided structure. This class uses instances of the `EnglishRule`, `FrenchRule`, and `ArabicRule` classes to apply language detection logic on each token. The implementation assumes that each rule class has a method (e.g., `matches(String token)`) that returns `true` if the token matches the language's characteristics.

java

```
package com.atx.langdetect.detector;

import com.atx.langdetect.rules.EnglishRule;
import com.atx.langdetect.rules.FrenchRule;
import com.atx.langdetect.rules.ArabicRule;
import com.atx.langdetect.util.LanguageDetectionException;

import java.util.List;
import java.util.HashMap;
import java.util.Map;

public class LanguageDetector {

    // Instances of the rule classes for each language
    private final EnglishRule englishRule;
    private final FrenchRule frenchRule;
    private final ArabicRule arabicRule;

    public LanguageDetector() {
        this.englishRule = new EnglishRule();
        this.frenchRule = new FrenchRule();
        this.arabicRule = new ArabicRule();
    }

    /**
     * Detects the language of a list of tokens.
     * @param tokens A list of text tokens to classify.
     * @return The detected language as a Language enum.
     * @throws LanguageDetectionException if detection fails.
     */
    public Language detectLanguage(List<String> tokens) throws LanguageDetectionException {
        // Map to store match counts for each language
        Map<Language, Integer> languageScores = new HashMap<>();
        languageScores.put(Language.ENGLISH, 0);
        languageScores.put(Language.FRENCH, 0);
        languageScores.put(Language.ARABIC, 0);

        // Analyze each token with the rules
        for (String token : tokens) {
            if (englishRule.matches(token)) {
                languageScores.put(Language.ENGLISH, languageScores.get(Language.ENGLISH) + 1);
            }
            if (frenchRule.matches(token)) {
                languageScores.put(Language.FRENCH, languageScores.get(Language.FRENCH) + 1);
            }
            if (arabicRule.matches(token)) {

```

```

        languageScores.put(Language.ARABIC, languageScores.get(Language.ARABIC) +
1);
    }

    // Determine the language with the highest score
    Language detectedLanguage = null;
    int maxScore = 0;
    for (Map.Entry<Language, Integer> entry : languageScores.entrySet()) {
        if (entry.getValue() > maxScore) {
            maxScore = entry.getValue();
            detectedLanguage = entry.getKey();
        }
    }

    if (detectedLanguage == null || maxScore == 0) {
        throw new LanguageDetectionException("Unable to determine the language from the
provided tokens.");
    }

    return detectedLanguage;
}
}

```

Explanation of Key Parts

1. Rule Instances:

- `EnglishRule`, `FrenchRule`, and `ArabicRule` instances are created and used to apply specific language detection rules.

2. `detectLanguage` Method:

- This method iterates over each token in the provided list.
- For each token, it checks if it matches the criteria for English, French, or Arabic by calling `matches(token)` on each rule instance.
- It increments a score for each language based on the number of matches found for that language.

3. Determining the Final Language:

- After counting matches, the method selects the language with the highest score.
- If no language scores any matches, it throws a `LanguageDetectionException`.

This code provides a foundation that leverages a basic rule-based system, ready to be refined as more specific rules are added to each rule class. Let me know if you'd like further customization!



****Enum: `Language`**** * Defines the supported languages: `ENGLISH`, `FRENCH`, and `ARABIC`. This enum serves as a classification result type for the detector.

Give me the "Language" enum, implementing this concepts, and tell the location, and use case of this logic in the project.



Here is the `Language` enum, which will be located in the `com.atx.langdetect.detector` package. This enum represents the supported languages and will be used as the return type in the `LanguageDetector`

class.

File Location

File Path: ``src/com/atx/langdetect/detector/Language.java``

``Language` Enum Code`

```
java

package com.atx.langdetect.detector;

public enum Language {
    ENGLISH,
    FRENCH,
    ARABIC;

    /**
     * Provides a user-friendly string representation of the language.
     * @return The name of the language in title case.
     */
    @Override
    public String toString() {
        switch (this) {
            case ENGLISH:
                return "English";
            case FRENCH:
                return "French";
            case ARABIC:
                return "Arabic";
            default:
                return super.toString();
        }
    }
}
```

Explanation and Use Case

1. Purpose:

- The ``Language`` enum defines the three languages that the detection system supports: ``ENGLISH``, ``FRENCH``, and ``ARABIC``.
- Using an enum ensures type safety and allows easy expansion by adding new languages if needed.

2. Location in Project:

- This enum resides in the ``detector`` package (``com.atx.langdetect.detector``), as it directly supports the detection logic.

3. Use Case in ``LanguageDetector``:

- The ``LanguageDetector`` class uses this enum as the return type for the ``detectLanguage`` method. The method returns one of the enum values (``Language.ENGLISH``, ``Language.FRENCH``, or ``Language.ARABIC``) based on which language had the highest score during the detection process.
- The enum's ``toString`` method provides a user-friendly representation of the detected language, which can be useful for display or logging purposes.

This setup makes it easy for other parts of the application to interpret and handle the detected language consistently and clearly. Let me know if you need further customization or additional details!



rules` Package** (`com.atx.langdetect.rules`): * **Purpose: Holds rule classes specific to each language, making it easier to add new languages by creating new rule classes in the future. *

Class: `EnglishRule` * Contains specific rules, patterns, or heuristics for detecting English text, such as checking for common English words or characters.

Give me the "EnglishRule.java" script as a sample of a class rule, implementing this concepts. The class should contains specific rules, patterns, or heuristics for detecting English text, such as checking for common English words or characters. You may apply the Regular Expression logics



Here is a sample implementation of `EnglishRule.java`. This class contains rule logic, using regex patterns to match common English words and characters. The `matches` method checks if a given token (a word or phrase) meets the criteria for being classified as English.

File Location

File Path: `src/com/atx/langdetect/rules/EnglishRule.java`

`EnglishRule.java` Code

```
java

package com.atx.langdetect.rules;

import java.util.regex.Pattern;

public class EnglishRule {

    // Regular expressions for common English patterns
    private static final Pattern COMMON_ENGLISH_WORDS = Pattern.compile("\\b(the|and|is|in|of|to|it|that|for|you|he|she|we)\\b", Pattern.CASE_INSENSITIVE);
    private static final Pattern ENGLISH_CHARACTERS = Pattern.compile("[a-zA-Z]+$");
    private static final Pattern ENGLISH_SENTENCE_STRUCTURE = Pattern.compile("[A-Z][a-z]*(\\s[a-z]+)*[.!?]?");

    /**
     * Checks if a given token matches English language patterns.
     * @param token The text token to evaluate.
     * @return true if the token matches English patterns; false otherwise.
     */
    public boolean matches(String token) {
        // Check if the token contains common English words
        if (COMMON_ENGLISH_WORDS.matcher(token).find()) {
            return true;
        }

        // Check if the token consists primarily of English alphabet characters
        if (ENGLISH_CHARACTERS.matcher(token).matches()) {
            return true;
        }

        // Check if the token follows a simple English sentence structure (capitalized start, lowercase words, and punctuation at the end)
        if (ENGLISH_SENTENCE_STRUCTURE.matcher(token).matches()) {
            return true;
        }
    }
}
```

```
// If no patterns matched, return false
return false;
}
```

Explanation of Key Parts

1. Regex Patterns:

- ``COMMON_ENGLISH_WORDS``: Matches common English words (e.g., "the", "and", "is"). This regex is case-insensitive, allowing it to match both uppercase and lowercase occurrences of these words.
- ``ENGLISH_CHARACTERS``: Checks if the token consists entirely of English alphabet characters (a-z or A-Z).
- ``ENGLISH_SENTENCE_STRUCTURE``: Matches tokens that resemble simple English sentence structures. It expects a capitalized first letter, followed by lowercase words, and an optional punctuation mark at the end.

2. ``matches`` Method:

- This method evaluates a token by applying the defined patterns. If the token matches any of these English-specific patterns, it returns ``true``, indicating that the token is likely English.
- If none of the patterns match, it returns ``false``.

Use Case in ``LanguageDetector``

The ``LanguageDetector`` class can create an instance of ``EnglishRule`` and call ``matches(token)`` for each token. If ``matches`` returns ``true``, it increments the score for English. This design allows the detection process to leverage rule-based heuristics specific to English, making it adaptable and easy to extend for other languages by following a similar pattern in rule classes.

Let me know if you'd like similar samples for the ``FrenchRule`` or ``ArabicRule`` classes!



****`util` Package**** (``com.atx.langdetect.util``): ****Purpose****: Houses utility and helper classes for common functions and error handling. ****Class: `TextUtils`**** * Provides utility methods for processing text, such as removing punctuation, converting text to lowercase, or splitting text into tokens if needed.

Give me the "TextUtils.java" script, implementing this concepts. Assuming the existence of all necessary classes (such as EnglishRule) and dependencies, tell the location, and use case of this logic in the project.



Here is the ``TextUtils.java`` class, which provides utility functions for text processing tasks. This class can be used to clean, tokenize, and preprocess text data before passing it to the ``LanguageDetector`` class for language detection.

File Location

File Path: `src/com/atx/langdetect/util/TextUtils.java`

`TextUtils.java` Code

```
java

package com.atx.langdetect.util;

import java.util.ArrayList;
import java.util.List;

public class TextUtils {

    /**
     * Removes punctuation from a given text string.
     * @param text The input string to process.
     * @return The text string without punctuation.
     */
    public static String removePunctuation(String text) {
        return text.replaceAll("\\p{Punct}", "");
    }

    /**
     * Converts a given text string to lowercase.
     * @param text The input string to convert.
     * @return The text string in lowercase.
     */
    public static String toLowerCase(String text) {
        return text.toLowerCase();
    }

    /**
     * Splits a given text into tokens (words) based on whitespace.
     * @param text The input string to split.
     * @return A list of tokens (words).
     */
    public static List<String> tokenize(String text) {
        String[] tokens = text.split("\\s+");
        List<String> tokenList = new ArrayList<>();
        for (String token : tokens) {
            if (!token.isEmpty()) {
                tokenList.add(token);
            }
        }
        return tokenList;
    }

    /**
     * Preprocesses a list of text tokens by removing punctuation and converting to lowercas
e.
     * @param tokens A list of text tokens to preprocess.
     * @return A list of cleaned tokens.
     */
    public static List<String> preprocessTokens(List<String> tokens) {
        List<String> processedTokens = new ArrayList<>();
        for (String token : tokens) {
            String cleanedToken = removePunctuation(token);
            cleanedToken = toLowerCase(cleanedToken);
            if (!cleanedToken.isEmpty()) {
                processedTokens.add(cleanedToken);
            }
        }
        return processedTokens;
    }
}
```

Explanation of Key Parts

1. Methods:

- `removePunctuation(String text)`: Removes punctuation characters from the input text using regex (`\p{Punct}`), which matches all punctuation marks.
- `toLowerCase(String text)`: Converts the input text to lowercase.
- `tokenize(String text)`: Splits the input text by whitespace to create a list of tokens (words).
- `preprocessTokens(List<String> tokens)`: Takes a list of tokens, removes punctuation, converts each token to lowercase, and returns a list of cleaned tokens.

Use Case in `LanguageDetector`

1. Token Preprocessing:

- Before passing tokens to the `LanguageDetector` for classification, the `TextUtils` methods can preprocess each token to ensure consistency. For example, `removePunctuation` and `toLowerCase` ensure that the detector receives uniformly formatted tokens.
- `TextUtils.tokenize` could be used in cases where an entire sentence or paragraph is input, breaking it down into individual tokens for more granular analysis.

2. Modularity:

- Keeping preprocessing logic in `TextUtils` allows for a modular approach. Any additional preprocessing steps (e.g., removing numbers) can be added in `TextUtils`, centralizing these functions for reuse across the project.

This utility class simplifies text handling, ensuring that the detection logic focuses solely on language identification without handling basic text transformations directly. Let me know if you'd like further customization!



****Class: `LanguageDetectionException`**** * A custom exception class to handle errors specific to language detection, like unsupported language tokens or invalid input data.
Give me the "LanguageDetectionException.java" script, implementing this concepts. Assuming the existence of all necessary and previously defined classes, methods, and dependencies, tell me the calling location (i.e. the files/classes that tend to call this logic), and use case of this logic in the project.



Here's the `LanguageDetectionException.java` class, a custom exception that handles specific errors encountered during language detection, such as unsupported languages or invalid input tokens.

File Location

File Path: `src/com/atx/langdetect/util/LanguageDetectionException.java`

`LanguageDetectionException.java` Code

```
java
```

```
package com.atx.langdetect.util;

/**
 * Custom exception for handling language detection errors.
 */
public class LanguageDetectionException extends Exception {

    /**
     * Constructs a new LanguageDetectionException with a specified detail message.
     * @param message The detail message.
     */
    public LanguageDetectionException(String message) {
        super(message);
    }

    /**
     * Constructs a new LanguageDetectionException with a specified detail message and cause.
     * @param message The detail message.
     * @param cause The cause of the exception.
     */
    public LanguageDetectionException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Explanation of Key Parts

1. Constructors:

- `LanguageDetectionException(String message)`: Allows creating an exception with a custom error message.
- `LanguageDetectionException(String message, Throwable cause)`: Allows including both a custom message and the underlying cause, enabling better debugging.

2. Inheritance:

- This class extends `Exception`, allowing it to be used as a checked exception. This makes it explicit when language detection encounters an issue, and callers are required to handle or declare it.

Use Case and Calling Location

Calling Location:

- **LanguageDetector Class:** The `LanguageDetectionException` is primarily called in `LanguageDetector.java`, specifically in the `detectLanguage` method. This method throws `LanguageDetectionException` when it cannot confidently determine the language of the tokens (e.g., if all languages score zero matches).

Example Usage in LanguageDetector:

```
java
```

```
public Language detectLanguage(List<String> tokens) throws LanguageDetectionException {
    // ... language detection logic ...
    if (detectedLanguage == null || maxScore == 0) {
        throw new LanguageDetectionException("Unable to determine the language from the provided tokens.");
    }
}
```

```

    }
    return detectedLanguage;
}

```

Use Case in the Project:

- **Purpose:** `LanguageDetectionException` provides a consistent way to report errors related to language detection issues. Instead of using generic exceptions, `LanguageDetectionException` allows the application to differentiate language detection errors from other types of errors, making debugging easier.
- **Error Handling:** Any class or method that calls `LanguageDetector.detectLanguage` can catch `LanguageDetectionException` and implement fallback logic, such as logging the error, notifying the user, or retrying with modified input data.

This setup ensures that language detection errors are easily identifiable, manageable, and consistent throughout the project. Let me know if you need further adjustments!



****Class: `FrenchRule`**** * Implements rules and patterns for detecting French, such as special characters (é, è, ç) or common French words (e.g., "le", "de").
Give me the "FrenchRule.java" script as a sample of a class rule, implementing this concepts. The class should contains specific rules, patterns, or heuristics for detecting French text, such as checking for common French words or characters. You may apply the Regular Expression logics.



Here's the `FrenchRule.java` class, which uses regex patterns to detect features specific to the French language. This class includes rules for identifying common French words, accents, and sentence structure typically seen in French text.

File Location

File Path: `src/com/atx/langdetect/rules/FrenchRule.java`

`FrenchRule.java` Code

```

java

package com.atx.langdetect.rules;

import java.util.regex.Pattern;

public class FrenchRule {

    // Regular expressions for common French patterns
    private static final Pattern COMMON_FRENCH_WORDS = Pattern.compile("\\b(le|la|de|un|une|et|ou|que|est|dans|avec|pour|ça)\\b", Pattern.CASE_INSENSITIVE);
    private static final Pattern FRENCH_ACCENTS = Pattern.compile(".*[éèëääâîïçôûù].*");
    private static final Pattern FRENCH_SENTENCE_STRUCTURE = Pattern.compile("^[A-Z][a-zââçéèêêîïôûüÿñæ]*(\\s[a-zââçéèêêîïôûüÿñæ]+)*[.!?]?$", Pattern.CASE_INSENSITIVE);

    /**

```

```

* Checks if a given token matches French language patterns.
* @param token The text token to evaluate.
* @return true if the token matches French patterns; false otherwise.
*/
public boolean matches(String token) {
    // Check if the token contains common French words
    if (COMMON_FRENCH_WORDS.matcher(token).find()) {
        return true;
    }

    // Check if the token contains French accent characters
    if (FRENCH_ACCENTS.matcher(token).matches()) {
        return true;
    }

    // Check if the token follows a typical French sentence structure
    if (FRENCH_SENTENCE_STRUCTURE.matcher(token).matches()) {
        return true;
    }

    // If no patterns matched, return false
    return false;
}
}

```

Explanation of Key Parts

1. Regex Patterns:

- `COMMON_FRENCH_WORDS`: Matches common French words (e.g., "le", "la", "de", "est"). This pattern is case-insensitive, allowing it to capture uppercase and lowercase occurrences.
- `FRENCH_ACCENTS`: Matches tokens containing accented characters commonly used in French (e.g., ``é`, `è`, `ç`, `à``). This regex ensures that if a token includes any of these characters, it's likely to be in French.
- `FRENCH_SENTENCE_STRUCTURE`: Matches tokens that resemble simple French sentence structure. This regex expects a capitalized start, lowercase words with accented characters allowed, and optional punctuation at the end.

2. `matches` Method:

- The `matches` method checks each token against the defined French-specific patterns. If the token matches any of these patterns, it returns `true`, indicating it is likely French.
- If none of the patterns match, it returns `false`.

Use Case in `LanguageDetector`

In the `LanguageDetector` class, an instance of `FrenchRule` can be used to check if each token matches French-specific features. If `matches(token)` returns `true`, the French score will be incremented for that token. This approach leverages targeted heuristics for French detection, making the classifier more effective without requiring complex models.

Let me know if you would like additional customization or explanations!



Class: `ArabicRule` * Contains logic for Arabic language detection, which might involve detecting Arabic script Unicode or other unique Arabic language patterns.

Give me the "ArabicRule.java" script as a sample of a class rule, implementing this concepts. The class should contains specific rules, patterns, or heuristics for detecting Arabic text, such as checking for common Arabic words or characters. You may apply the Regular Expression logics.



Here's the `ArabicRule.java` class, which uses regex patterns and heuristics to identify Arabic text. This class includes rules for detecting Arabic script Unicode characters, common Arabic words, and punctuation patterns typical in Arabic text.

File Location

File Path: `src/com/atx/langdetect/rules/ArabicRule.java`

`ArabicRule.java` Code

```

java
package com.atx.langdetect.rules;

import java.util.regex.Pattern;

public class ArabicRule {

    // Regular expressions for common Arabic patterns
    private static final Pattern COMMON_ARABIC_WORDS = Pattern.compile("\\b(ل|و|من|في|على|م|ع|هـ|ها|هذا|ذلك|ذلك|ع)\\b");
    private static final Pattern ARABIC_CHARACTERS = Pattern.compile("[\\u0600-\\u06FF]");
    private static final Pattern ARABIC_SENTENCE_STRUCTURE = Pattern.compile("^([\\u0600-\\u06FF\\s]+[.,?]?$)");

    /**
     * Checks if a given token matches Arabic language patterns.
     * @param token The text token to evaluate.
     * @return true if the token matches Arabic patterns; false otherwise.
     */
    public boolean matches(String token) {
        // Check if the token contains common Arabic words
        if (COMMON_ARABIC_WORDS.matcher(token).find()) {
            return true;
        }

        // Check if the token contains Arabic script characters
        if (ARABIC_CHARACTERS.matcher(token).find()) {
            return true;
        }

        // Check if the token follows a simple Arabic sentence structure
        if (ARABIC_SENTENCE_STRUCTURE.matcher(token).matches()) {
            return true;
        }

        // If no patterns matched, return false
        return false;
    }
}

```

Explanation of Key Parts

1. Regex Patterns:

- ``COMMON_ARABIC_WORDS``: Matches frequently used Arabic words, such as "ال" (the definite article), "و" (and), "في" (in), etc. These are common across various Arabic texts and can indicate that a token is likely in Arabic.
- ``ARABIC_CHARACTERS``: Matches any character in the Arabic Unicode block (``\u0600-\u06FF``), which covers all basic Arabic letters, numbers, and diacritical marks.
- ``ARABIC_SENTENCE_STRUCTURE``: Matches typical Arabic sentence structures, where tokens are composed of Arabic script characters with optional Arabic punctuation (such as ``،`` for comma or ``؟`` for question mark).

2. ``matches`` Method:

- The ``matches`` method evaluates whether a given token follows Arabic-specific patterns by checking against each regex pattern. If any pattern matches, it returns ``true``, identifying the token as likely Arabic.
- If none of the patterns match, it returns ``false``.

Use Case in ``LanguageDetector``

In ``LanguageDetector.java``, an instance of ``ArabicRule`` can be created to analyze each token and check for Arabic characteristics. If ``matches(token)`` returns ``true``, the Arabic language score for that token will be incremented. This structured approach enhances accuracy for Arabic detection by combining rule-based language heuristics tailored for Arabic script.

Let me know if you'd like further refinement or additional examples!



Now, Assuming that all the project files are defined, including the packages `/detector`, `/rules`, and `/util`, it is essential to combine all the classes' concepts into a single file logic; the "LangDetectMain.java" class. This global main class should then be treated as an entry point for a typical use of the language detection mechanism.

After the completion of all the files (e.g. classes, methods, dependencies, ...etc.), and all the project components, give me the "LangDetectMain.java" script, implementing and combining all those concepts and logic, initializing the language detection and providing an entry point to the project.



Here's the ``LangDetectMain.java`` file, which serves as the main entry point for the language detection project. This file initializes all the required classes, invokes the language detection mechanism, and provides an example of how to use the ``LanguageDetector`` with sample input.

File Location

File Path: ``src/com/atx/langdetect/LangDetectMain.java``

``LangDetectMain.java`` Code

```
java

package com.atx.langdetect;
```

```
import com.atx.langdetect.detector.LanguageDetector;
import com.atx.langdetect.rules.EnglishRule;
import com.atx.langdetect.rules.FrenchRule;
import com.atx.langdetect.rules.ArabicRule;
import com.atx.langdetect.util.LanguageDetectionException;
import com.atx.langdetect.util.TextUtils;

import java.util.Arrays;
import java.util.List;

public class LangDetectMain {

    public static void main(String[] args) {
        // Sample text inputs for language detection
        List<String> textSamples = Arrays.asList(
            "Hello, how are you?",
            "Bonjour, comment ça va?",
            "مرحبا كيف حاله؟"
        );

        // Initialize the language detector with rule-based classes
        LanguageDetector detector = new LanguageDetector(new EnglishRule(), new FrenchRule(), new ArabicRule());

        // Process each sample and detect its language
        for (String text : textSamples) {
            try {
                // Preprocess the input text
                List<String> tokens = TextUtils.tokenize(text);
                List<String> processedTokens = TextUtils.preprocessTokens(tokens);

                // Detect language
                Language detectedLanguage = detector.detectLanguage(processedTokens);
                System.out.println("Input: \"" + text + "\" is detected as: " + detectedLanguage);
            } catch (LanguageDetectionException e) {
                System.err.println("Error detecting language for input: \"" + text + "\"");
                e.printStackTrace();
            }
        }
    }
}
```

Explanation of Key Parts

1. Initialization:

- The `LanguageDetector` object is created with instances of `EnglishRule`, `FrenchRule`, and `ArabicRule`. This configures the detector to handle English, French, and Arabic languages using the defined rule classes.

2. Token Processing:

- For each input sentence, `TextUtils.tokenize(text)` is called to split the sentence into tokens (words). Then, `TextUtils.preprocessTokens(tokens)` cleans each token by removing punctuation and converting it to lowercase.

3. Language Detection:

- The `detectLanguage(processedTokens)` method of `LanguageDetector` is called with the preprocessed tokens. This method returns a `Language` enum value representing the detected language (English, French, or Arabic).
- If language detection fails (e.g., if the tokens don't match any language rules), a `LanguageDetectionException` is thrown, caught, and logged to `System.err`.

4. Output:

- For each sample text, the detected language is printed to the console, showing which language (English, French, or Arabic) was identified for the given input.

Example Output

When ``LangDetectMain`` is run, the output could look like:

```
plaintext
```

```
Input: "Hello, how are you?" is detected as: ENGLISH
Input: "Bonjour, comment ça va?" is detected as: FRENCH
Input: "مرحبا كيف حالك؟" is detected as: ARABIC
```

Use Case

``LangDetectMain`` provides a clear entry point to test the entire language detection process with a variety of input samples. It showcases the following:

- Initialization and configuration of ``LanguageDetector`` with rule-based detection classes (``EnglishRule``, ``FrenchRule``, ``ArabicRule``).
- Preprocessing using ``TextUtils``, allowing consistent and standardized token preparation.
- Exception handling through ``LanguageDetectionException``, giving meaningful feedback when detection fails.

This setup integrates all project components cohesively, allowing for effective language detection based on rule-based heuristics. Let me know if you need further enhancements or examples!